

MBLP136 – VERİ TABANI

6. Hafta: Vize Öncesi Genel Tekrar (1. - 5. Haftalar)

Dr. Öğr. Üyesi Hasan Oğuz

17 Mart 2026

İstanbul Okan Üniversitesi
Meslek Yüksekokulu

Temel Kavramlar: Veri ve Sistem

Veri Tabanı (Database) Nedir?

- **Veri (Data):** İşlenmemiş, ham gerçeklerdir (Örn: "105", "Kırmızı", "3.14"). Kendi başlarına analitik bir değer taşımazlar.
- **Bilgi (Information):** Verinin belirli bir bağlam (context) içinde işlenmiş ve anlamlandırılmış halidir.
- **Veri Tabanı:** Birbiriyle mantıksal olarak ilişkili verilerin, belirli bir şema (schema) dahilinde, bilgisayar ortamında sistematik olarak depolandığı dijital yapıdır.

Teknik Tanım

Veri tabanı, verilerin sadece depolandığı pasif bir yığın değil; veriler arasındaki bağıntıların ($R \subseteq A \times B$) ve yapısal kısıtlamaların (constraints) aktif olarak korunduğu dinamik bir sistemdir.

Neden Veri Tabanı Kullanıyoruz?

Geleneksel dosya sistemlerinin (Örn: Excel, Flat-file yapıları) limitasyonlarını aşmak için VTYS (Veri Tabanı Yönetim Sistemleri) kullanırız:

- **Veri Bütünlüğü (Data Integrity):** Tutarsızlıkları ve gereksiz veri tekrarlarını (redundancy) fiziksel düzeyde önler.
- **Eşzamanlı Erişim (Concurrency):** Binlerce kullanıcının aynı anda, birbirinin verisini ezmeden (ACID izolasyonu ile) işlem yapabilmesini sağlar.
- **Ölçeklenebilirlik (Scalability):** Veri boyutu terabaytlara ulaştığında bile performansın doğrusal olarak düşmesini engeller.
- **Güvenlik (Security):** Kullanıcı bazlı yetkilendirme ile "kimin, hangi veriyi görebileceğini" matematiksel olarak sınırlar.

Neden Verileri Sınıflandırma İhtiyacımız Var?

Sınıflandırma ve yapılandırma, sistemdeki **veri entropisini** (kaosu) düşürmenin ve deterministik bir yapı kurmanın tek yoludur:

- **Performans (Erişilebilirlik):** Verileri belirli özniteliklere göre sınıflandırmak, indeksleme (B-Tree) mimarilerini kullanarak milyonlarca kayıt arasından milisaniyeler içinde arama yapmamızı sağlar.
- **Atomik Yapı (1NF):** Veriyi en küçük, bölünemez parçalarına ayırarak güncelleme ve silme anomalilerini (hatalarını) engelleriz.
- **İlişkisel Bütünlük:** Bir verinin (Örn: Bölüm) diğer verilerle (Örn: Öğrenci) olan nedensel ve mantıksal bağıını, sınıflandırma sayesinde Yabancı Anahtarlar (FK) ile modelleyebiliriz.

Sonuç: Yapılandırılmamış veri bir "gürültü" (noise), sınıflandırılmış veri ise bir "sinyal" (signal) taşır.

Mimari

ANSI-SPARC Mimarisi: Katmanlar Arası Eşleme (Mapping)

Üç seviyeli mimarinin temel fonksiyonu, katmanlar arasındaki dönüşümü sağlayan **Eşleme (Mapping)** mekanizmasıdır. Bu, karmaşıklığın izole edilmesini sağlar:

- **Dış/Kavramsal Eşleme ($\Phi_{E \rightarrow C}$):** Kullanıcının gördüğü projeksiyonların (V_n), ana mantıksal şema (S_C) ile nasıl ilişkilendirildiğini tanımlar. Farklı kullanıcı gruplarına aynı verinin farklı görünüşleri sunulur.
- **Kavramsal/İç Eşleme ($\Psi_{C \rightarrow I}$):** Mantıksal kayıtların disk üzerindeki fiziksel bloklara, indeksleme yapılarına (B^+ -Tree, Hash) veya sıkıştırma algoritmalarına nasıl dönüştürüldüğünü belirler.

Sistem Soyutlama Fonksiyonu

Veri erişim süreci, katmanlar arası bir fonksiyonel dönüşüm zinciri olarak modellenenebilir:

$$Query_{User} \xrightarrow{\Phi^{-1}} Query_{Conceptual} \xrightarrow{\Psi^{-1}} Query_{Physical}$$

Veri Bağımsızlığının Taksonomisi

Veri bağımsızlığı, sistemin sürdürülebilirliği (maintainability) açısından iki ana kategoride incelenir:

1. **Fiziksel Veri Bağımsızlığı:** İç seviyedeki (ΔS_I) değişikliklerin (örneğin depolama ortamının SSD'den NVMe'ye geçmesi veya yeni bir *index* eklenmesi), kavramsal şemayı (S_C) etkilememesidir. Sadece $\Psi_{C \rightarrow I}$ eşlemesi güncellenir.
2. **Mantıksal Veri Bağımsızlığı:** Kavramsal seviyedeki (ΔS_C) yapısal değişikliklerin (yeni bir tablo eklenmesi veya mevcut tabloların bölünmesi), dış seviye görünümüleri (V_n) ve uygulama kodlarını etkilememesidir.

Spekülasyon: Geleceğin DBMS mimarilerinde, *Self-Tuning Mapping* algoritmaları sayesinde Ψ eşlemesinin AI tarafından çalışma zamanında (runtime) optimize edilmesi, $n(r)$ gradyan indeksli optik sistemlerdeki ışık bükülmesine benzer şekilde veri erişim yollarını dinamik olarak "refrakte" edebilir.

Veri Yönetiminde ACID Prensiplerine Giriş

Veritabanı işlemlerinde (Transaction), sistemin bir tutarlı durumdan (S_0) diğerine (S_1) güvenli geçişini sağlayan temel protokoller bütünüdür. Bir işlem $\mathcal{T}$, atomik olarak değerlendirilen bir operasyon dizisidir.

- **Tanım:** $\mathcal{T} = \{o_1, o_2, \dots, o_n\}$ kümesindeki tüm operasyonların bütünlüğü korunmalıdır.
- **Motivasyon:** Donanım arızaları, ağ kesintileri veya eşzamanlı (concurrent) erişimler altında veri bütünlüğünü (*Data Integrity*) garanti etmek.

RDBMS vs NoSQL Paradigması

RDBMS sistemleri katı ACID kurallarıyla *Pessimistic Concurrency Control* sağlarken, dağıtık NoSQL sistemler genellikle CAP teoremi gereği *Eventual Consistency* (BASE) modeline yönelir.

Atomicity (Bölünemezlik): "Ya Hep Ya Hiç"

İşlem içerisindeki adımların bir bütün olarak ele alınmasıdır. Eğer $\mathcal{T}$ içindeki herhangi bir o_i başarısız olursa, tüm işlem **Rollback** edilerek sistem S_0 başlangıç durumuna döner.

Örnek: Banka Havalesi

Hesap A'dan 100 TL düşülmesi (o_1) ve Hesap B'ye 100 TL eklenmesi (o_2).

- Eğer o_1 tamamlanır ancak sistem o_2 öncesi çökerse, o_1 geri alınmalıdır.
- Mekanizma: **Write-Ahead Logging (WAL)** veya **Shadow Paging**.

Teknik Not: Fiziksel katmanda bu durum, disk yazma kafasının atomik yazma birimi (sector) boyutuyla sınırlı olmaması için log tabanlı yönetim gerektirir.

Consistency (Tutarlılık): Kısıtların Korunması

İşlem tamamlandığında veritabanının önceden tanımlanmış tüm iş kurallarına (Constraints), anahtarlara (Foreign Keys) ve tetikleyicilere (Triggers) uygun kalmasıdır.

- **Invariant:** Bir veritabanı durumu S , $C(S) = true$ koşulunu sağlamalıdır.
- İşlem sonrası: $S_0 \xrightarrow{\mathcal{T}} S_1 \implies C(S_1)$ geçerli olmalıdır.

Örnek: Stok Kontrolü

Bir e-ticaret sisteminde stok miktarı $Qty \geq 0$ kısıtı varsa; bir satış işlemi sonucunda stok negatif oluyorsa, veritabanı motoru işlemi *Consistency Violation* nedeniyle reddeder.

Isolation (İzolasyon): Eşzamanlılık Kontrolü

Birden fazla işlemin ($\mathcal{T}_1, \mathcal{T}_2, \dots$) aynı anda çalışırken birbirlerinin ara durumlarını görmemesini sağlar. Bu, **Concurrency Control** algoritmaları ile yönetilir.

- **Seviyeler:** Read Uncommitted, Read Committed, Repeatable Read, Serializable.
- **Problemler:** Dirty Read, Non-repeatable Read, Phantom Read.

Uygulama: MVCC ve Kilitleme

Modern sistemler (PostgreSQL vb.), verinin farklı versiyonlarını tutarak (*Multi-Version Concurrency Control*) okuma işlemlerinin yazma işlemlerini engellemesini önler.

Durability (Kalıcılık): Sistemsel Kararlılık

Bir işlem kullanıcıya "başarılı" (Commit) olarak bildirildikten sonra, sistem çökse, elektrik kesilse veya donanım arızası yaşansa dahi verinin kalıcı olarak saklanması garantisidir.

- **Mekanizma:** Veri ana veri dosyalarına (*MDF*) yazılmadan önce, hızlı erişilen bir **Transaction Log** (*LDF*) dosyasına kaydedilir.
- **Checkpoint:** Bellekteki kirli sayfaların (dirty pages) belirli aralıklarla diske kalıcı olarak senkronize edilmesi işlemidir.

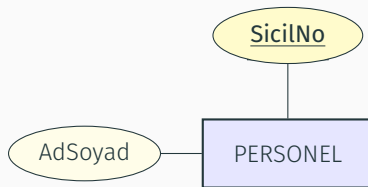
Spekülasyon: NVMe ve Persistent Memory (PMEM) teknolojilerinin gelişimi, klasik WAL protokollerini gereksiz kılarak doğrudan bellek üzerinde atomik güncellemelerin önünü açabilir; bu da veritabanı gecikmelerini (latency) nanosaniye seviyelerine indirebilir.

ER Modeli ve İlişkisel Haritalama

Kavramsal Tasarım: Varlıklar ve Temel Öznitelikler

Veritabanı tasarımının ontolojik temelini oluşturan varlıklar ve onların atomik özellikleri, küme teorisi prensiplerine göre haritalanır:

- **Varlık (Entity):** Sistemde bağımsız olarak var olan, diğer nesnelerden ayırt edilebilen kavramlar. Sembol: **Dikdörtgen**.
- **Öznitelik (Attribute):** Bir varlığın sahip olduğu karakteristik veriler. Sembol: **Elips**.
- **Anahtar Öznitelik (Primary Key):** Varlık kümesindeki ($\mathcal{E}$) her bir elemanı tekil olarak tanımlayan, fonksiyonel bağımlılığın ($\mathcal{PK} \rightarrow \mathcal{A} \sqcup \nabla \rangle [\sqcup] f$) köküdür. Sembol: **Altı çizili elips**.



ER Sembolojisi: Kompleks Öznitelikler ve İlişkiler

Veri modelinde dinamik hesaplamaları ve çoklu bağıntıları ifade etmek için özelleşmiş semboller kullanılır:

- **Çok Değerli (Multivalued):** Bir varlık örneği için birden fazla veri girişi yapılabilecek alanlar (Örn: {Uzmanlık Alanları}). Sembol: **Çift Elips**.
- **Türetilen (Derived):** Fiziksel olarak saklanmayan, diğer öznitelikler üzerinden hesaplanan değerler. Sembol: **Kesikli Elips**.
- **İlişki (Relationship):** Varlıklar arasındaki mantıksal ve matematiksel bağıntıyı temsil eder. Sembol: **Baklava Dilimi**.

Matematiksel Formülasyon

Türetilen öznitelik bir projeksiyon fonksiyonudur: $\phi : S_{Internal} \rightarrow V_{External}$. Örn:

$Yaş = \Delta t(Bugün, DoğumTarihi)$.

Tahmin: Geleceğin *Self-Healing Schema* mimarilerinde, ER diyagramları birer *Knowledge Graph* olarak işlenecek ve normalizasyon hataları "Manifold Learning" teknikleriyle otomatik olarak minimize edilecektir.

İlişkisel Haritalama: 1 : 1 ve 1 : N Bağlantıları

Varlık-İlişki (ER) modelinden mantıksal şemaya geçişte, kardinalite kısıtları veri fazlalığını (*Redundancy*) önlemek için belirli topolojiler dikte eder:

- 1 : 1 (**Bire-Bir**): Her iki varlık kümesi de tam katılımlıysa (*Total Participation*), tablolar birleştirilebilir. Kısmi katılımda, *Foreign Key* (FK) genellikle opsiyonelliği az olan tarafa yerleştirilir.
- 1 : N (**Bire-Çok**): "Bire" karşılık "Çok" ($\mathcal{E}_1 \rightarrow \mathcal{E}_N$) ilişkisinde, 1 tarafındaki birincil anahtar (*PK*), *N* tarafındaki tabloya bir sütun (*FK*) olarak eklenir.

Fonksiyonel Bağımlılık Karakterizasyonu

1 : N ilişkisi, matematiksel olarak $N \rightarrow 1$ yönünde bir fonksiyonel bağımlılık oluşturur:

$$f : \text{Attributes}(\mathcal{E}_N) \rightarrow PK(\mathcal{E}_1)$$

Kompleks İlişkiler: $M : N$ Çözümlemesi ve Ara Tablolar

Çoğa-çok ($M : N$) ilişkiler, atomik veri yapısını korumak adına doğrudan birincil/yabancı anahtar çiftiyle modellenemez. Bu durum, Birinci Normal Form (1NF) ihlaline yol açar.

- **Ara Tablo (Junction/Associative Table):** İlişkinin kendisi bir tabloya dönüştürülür. Bu tablo, her iki ana tablonun PK 'lerini FK olarak bünyesinde barındırır.
- **Bileşik Anahtar (Composite Key):** Ara tablonun birincil anahtarı genellikle bu iki FK 'nın kombinasyonundan oluşur: $PK_{Ara} = \{FK_A, FK_B\}$.

Örnek: Ders Kayıt Sistemi

Öğrenci(M) $\leftrightarrow$ Ders(N) ilişkisinde, "Kayıt" tablosu oluşturulur.

Speculation: Modern *Graph-Relational Hybrid* sistemlerde, $M : N$ ilişkileri fiziksel katmanda gizli *pointer* dizileriyle (refractive index mapping benzeri bir yönlendirme ile) yönetilerek *JOIN* maliyeti $O(1)$ karmaşıklığına indirgenebilir.

Normalizasyon Teorisi

Normalizasyon: Veri Sistemlerinde Entropi Minimizasyonu

Veritabanı normalizasyonu, bir sistemin *redundancy* (veri tekrarı) kaynaklı potansiyel enerjisini düşürerek **Ground State** (kararlı durum) yapısına ulaştırılması sürecidir. Amaç, veri anomalilerini ortadan kaldırmaktır.

1NF (Birinci Normal Form): Atomisite ve Kuantizasyon

- **Tanım:** Tüm öznitelikler ($\mathcal{A}$) atomik olmalı; yani $\forall a \in \mathcal{A}$, $val(a)$ bir skaler değer olmalıdır.
- **Kısıt:** Tekrarlayan gruplar ve çok değerli öznitelikler (*multivalued attributes*) tablo yapısından arındırılır.

Anomali Örneği

Bir hücrede {Fizik, Optik, Kuantum} gibi bir küme tutulması, ilişkisel cebir operatörlerinin (σ, π) seçiciliğini bozar ve veri entropisini artırır.

2NF: Kısmi Bağımlılıkların Eliminasyonu

Bir tablonun 2NF olması için 1NF şartını sağlaması ve tüm anahtar olmayan özniteliklerin, birincil anahtarın ($\mathcal{PK}$) **tamamına** fonksiyonel olarak bağımlı olması gerekir.

- **Partial Dependency (Kısmi Bağımlılık):** Eğer $\mathcal{PK} = \{A, B\}$ ise ve bir öznitelik C için $A \rightarrow C$ sağlanıyorsa (fakat B gerekmiyorsa), bu bir 2NF ihlalidir.
- **Çözüm:** Kısmi bağımlılığa neden olan öznitelikler, belirleyici (*determinant*) anahtar parçası ile birlikte yeni bir tabloya taşınır.

Fiziksel Analoji

Bu durum, sistemdeki bir serbestlik derecesinin ($\mathcal{C}$) sadece bir alt-parametreye ($\mathcal{A}$) bağlı olup, toplam Hamiltonian'dan ($\mathcal{PK}$) bağımsız hareket edebilmesine benzer; sistemin ayrıştırılması (decoupling) gerekir.

3NF: Geçişli Bağımlılıklar ve Doğrudan Erişim

3NF, 2NF koşullarına ek olarak, anahtar olmayan öznitelikler arasında hiçbir fonksiyonel bağımlılığın bulunmamasını gerektirir.

- **Transitive Dependency (Geçişli Bağımlılık):** Eğer $PK \rightarrow A$ ve $A \rightarrow B$ ise, dolaylı yoldan $PK \rightarrow B$ gerçekleşir. Bu, verinin A üzerinden B'yi belirlemesi nedeniyle fazlalık yaratır.
- **Kriter:** Anahtar olmayan her sütun; "Anahtara, bütün anahtara ve anahtardan başka hiçbir şeye" bağımlı olmalıdır (Codd'un vasiyeti).

Normalizasyon ve Performans Trade-off

Yüksek normalizasyon seviyeleri (3NF, BCNF) *Join* maliyetini artırır. Analitik sistemlerde (OLAP) bazen performansı artırmak için kontrollü **Denormalizasyon** uygulanır.

Speculation: Gelecekteki "Topological Data Analysis" (TDA) tabanlı DB motorları, şema bağımlılıklarını *Simplicial Complex* olarak modelleyerek 3NF ihlallerini manifold üzerindeki süreksizlikler olarak saptayabilir.

Örnek Sorular

Sınav Hazırlık: ANSI-SPARC ve Veri Bağımsızlığı

1. ANSI-SPARC mimarisinde, veritabanı yöneticisinin (DBA) disk üzerindeki indeksleme yapısını B^+ -Tree'den Hash-Index'e dönüştürmesi hangi seviyede gerçekleşir ve bu durum uygulama kodlarını neden etkilemez?
2. Fiziksel veri bağımsızlığı ile mantıksal veri bağımsızlığı arasındaki temel farkı, katmanlar arası eşlemeler ($\Phi_{E \rightarrow C}$ ve $\Psi_{C \rightarrow I}$) üzerinden açıklayınız.
3. Kavramsal seviyenin (Logical Level) "kurumsal veri modeli" olarak adlandırılmasının ardındaki temel motivasyon nedir?
4. Bir veritabanı sisteminde "Görünüm" (View) katmanının kullanılmaması, ANSI-SPARC mimarisinin hangi temel ilkesini ihlal eder?

Sınav Hazırlık: ACID Prensipleri

1. Bir bankacılık işleminde, kaynak hesaptan para düşülüp hedef hesaba eklenmeden sistemin çökmesi durumunda ACID'in hangi prensibi devreye girer? Bu durumun fiziksel çözümünde kullanılan **Write-Ahead Logging (WAL)** mekanizmasını açıklayınız.
2. "İzolasyon" (Isolation) seviyelerinden *Serializable* ile *Read Committed* arasındaki farkın, eşzamanlı (concurrent) işlemler üzerindeki etkisini tartışınız.
3. Veritabanı kısıtlarının (Foreign Key, Check Constraints) "Tutarlılık" (Consistency) prensibi ile olan matematiksel ilişkisini $C(S_0) \rightarrow C(S_1)$ geçişi üzerinden modelleyiniz.
4. Başarıyla *Commit* edilmiş bir işlemin, disk arızası durumunda bile korunmasını sağlayan "Kalıcılık" (Durability) mekanizması, donanım katmanında nasıl bir "Redundancy" gerektirir?

Sınav Hazırlık: ER Modelleme ve Semboloji

1. Bir ER diyagramında "Personel" varlığına bağlı "Yaş" özniteliğinin neden kesikli elips (Derived Attribute) olarak modellenmesi gerektiğini, veri bütünlüğü ve depolama optimizasyonu açısından değerlendiriniz.
2. Çok değerli (Multivalued) özniteliklerin (Örn: Bir personelin birden fazla uzmanlık sertifikası olması) ilişkisel şemaya aktarılırken neden ayrı bir tabloya dönüştürülmesi zorunludur?
3. Zayıf Varlık (Weak Entity) kümesinin, birincil anahtara sahip olmamasından kaynaklanan "Bağımlı Varlık" (Identifying Relationship) kavramını ER sembolojisiyle nasıl ifade edersiniz?
4. İlişki (Relationship) baklava dilimlerinin kendi özniteliklerine sahip olması (Örn: "Satın Alır" ilişkisindeki "Tarih" bilgisi) ilişkisel modelde nereye haritalanır?

Sınav Hazırlık: Kardinalite ve Mapping Algoritmaları

1. $1 : N$ bir ilişkide, "1" tarafındaki anahtarın "N" tarafına yabancı anahtar (FK) olarak taşınmasının matematiksel gerekliliğini, fonksiyonel bağımlılıklar üzerinden kanıtlayınız.
2. Bir üniversite sistemindeki $M : N$ (Öğrenci-Ders) ilişkisini, bir ara tablo (Junction Table) kullanmadan doğrudan modellemeye çalışmanın yaratacağı "Anomali" durumlarını açıklayınız.
3. İlişkisel şemaya geçişte (Relational Mapping), bir ara tablonun birincil anahtarı (PK) genellikle nasıl oluşturulur? Bileşik anahtar (*Composite Key*) kullanımı burada neden kritiktir?
4. "Kardinalite Oranı" (Cardinality Ratio) ile "Katılım Kısıtı" (Participation Constraint - Total/Partial) arasındaki farkı, veritabanı şemasındaki *NULL* değer oluşumu üzerinden analiz ediniz.

Sınav Hazırlık: Normalizasyon Teorisi

1. Bir tablonun $1NF$ olup $2NF$ olmaması durumunda karşılaşılan "Kısmi Bağımlılık" (Partial Dependency) sorununu, birleşik anahtarlı bir tablo örneği üzerinden demonstre ediniz.
2. $3NF$ seviyesinde bir tablonun "Geçişli Bağımlılık" (Transitive Dependency) içermemesi kuralı, veritabanı güncellemelerinde (Update Anomaly) bize nasıl bir avantaj sağlar?
3. Normalizasyon işleminin "Veri Entropisini Düşürmek" olarak tanımlanması durumunda; yüksek normalizasyonun (BCNF ve ötesi) *Query Performance* üzerindeki negatif etkilerini tartışınız.
4. Verilen bir $R(A, B, C, D)$ şemasında, $A \rightarrow B$ ve $B \rightarrow C$ bağımlılıkları mevcutsa, bu tablonun $3NF$ olması için hangi ayrıştırma (Decomposition) işlemlerinin yapılması gerekir?

Sınav Stratejisi ve Kontrol Listesi

Arasınay Kontrol Listesi

Bu 5 haftalık sürecin sonunda sınavda ölçülecek temel yetkinlikleriniz:

- ☐ **Teori:** Veri, Bilgi ve Veri Tabanı nedir biliyorum.
- ☐ **Teori:** Neden veri Tabanına ihtiyacımız olduğunu, neden verileri sınıflandırmaya ihtiyaç duyduğumuzu anlıyorum.
- ☐ **Teori:** ACID prensiplerini ve ANSI-SPARC mimarisini tanımlayabiliyorum.
- ☐ **Kavramsal:** Verilen bir senaryodan (Örn: Kütüphane) ER diyagramı çizebiliyor; PK, çok değerli ve türetilen öznitelikleri doğru sembollerle gösterebiliyorum.
- ☐ **Mantıksal:** Çizdiğim ER diyagramını, M:N ilişkileri "Ara Tablo" ile çözecek şekilde ilişkisel şemalara dönüştürebiliyorum.
- ☐ **Optimizasyon:** Karmaşık bir Excel tablosundaki veri tekrarlarını tespit edip, 3NF kurallarına göre tabloları parçalayabiliyorum.

Başarılar Dilerim.